

RunuX: A Formally Verified, FFI-Compatible Rust Translation of the Linux Kernel Networking Stack

Xavier Callens
callensxavier@gmail.com
Independent Researcher
Brussels, Belgium

ABSTRACT

Memory safety vulnerabilities remain the dominant class of critical security flaws in operating system kernels. We present **RunuX**, the first comprehensive, FFI-compatible Rust translation of 297 core Linux kernel networking modules, targeting the Linux 5.10 LTS codebase. RunuX achieves 100% compilation coverage with zero type-integrity warnings through a unified `kernel_types` crate providing binary-compatible `#[repr(C)]` structures. We validate run-time correctness through (i) automated QEMU boot testing, (ii) continuous libFuzzer-based packet injection, and (iii) sustained fault injection on Google Kubernetes Engine using Chaos Mesh. Across six distinct fault scenarios—network partition, latency injection, packet loss, CPU starvation, memory pressure, and pod eviction—RunuX exhibits **zero kernel panics** and **zero oops**. Performance evaluation shows Rust CRC32 checksum throughput exceeding the C baseline by 4.73%, with QEMU boot time within 0.02% of the native C kernel. Formal verification using Lean 4 spans 432 theorems and 138 axioms across 36 specification modules; a dedicated Ring 0 isolation model (RunuXDefenses, 84 theorems) is fully closed with zero sorry tactics and zero axioms, while the broader network-stack modules retain 245 open proof obligations, continuously tracked by an automated metrics gate. We additionally report a first pilot of an oracle-gated, multi-tier LLM proof-completion workflow that closed 4 of 16 targeted theorems with independently re-verified proofs and correctly flagged 2 further theorems as unprovable-as-stated—genuine specification defects—rather than fabricating unsound proofs. RunuX demonstrates that a production-grade, memory-safe kernel translation is feasible without sacrificing performance or binary compatibility, and that closing the remaining proof debt can be partially automated under a strict, independently-checked contract.

CCS CONCEPTS

• **Software and its engineering** → **Language features**; • **Computer systems organization** → **Embedded and cyber-physical systems**.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

EuroSys '27, 2027, Rome, Italy

© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/27/03
<https://doi.org/10.1145/XXXXXXX.XXXXXXX>

KEYWORDS

Rust, Linux Kernel, Formal Verification, Memory Safety, FFI, Chaos Engineering, Lean 4, Verus

ACM Reference Format:

Xavier Callens. 2027. RunuX: A Formally Verified, FFI-Compatible Rust Translation of the Linux Kernel Networking Stack. In *Proceedings of The Eighteenth European Conference on Computer Systems (EuroSys '27)*. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/XXXXXXX.XXXXXXX>

1 INTRODUCTION

For over three decades, the Linux kernel has served as the foundational infrastructure of modern computing—powering servers, embedded devices, mobile phones, and cloud platforms. However, its implementation in C has inherently exposed it to memory safety vulnerabilities. A 2021 analysis by Google [1] found that approximately 70% of high-severity security bugs in Chrome and Android are memory safety issues. Microsoft reported similar findings for Windows [2].

The Rust programming language offers a compelling alternative: compile-time memory safety guarantees through ownership, borrowing, and lifetime checking—without garbage collection overhead. The Linux kernel community has recognized this potential, with initial Rust-for-Linux patches merged into the mainline kernel as of Linux 6.1 [3]. However, these efforts have been limited to isolated driver modules.

RunuX takes a fundamentally different approach. Rather than writing new modules in Rust, we perform a *systematic, structure-preserving translation* of the entire Linux networking stack—297 modules spanning IPv4/IPv6, Netfilter/NAT, TCP/UDP/ICMP, routing, tunneling, and socket layers—while maintaining exact binary compatibility with the C ABI through `#[repr(C)]` annotations.

1.1 Acknowledgments

We extend our deepest gratitude to **Linus Torvalds** and the broader **Linux kernel community**. RunuX does not seek to replace Linux, but rather to stand on the shoulders of giants. The decades of meticulous engineering, architectural brilliance, and open-source dedication poured into the Linux kernel are what made RunuX possible. This project is a tribute to that legacy, offering a memory-safe pathway forward for the most critical infrastructure on Earth.

1.2 Contributions

This paper makes the following contributions:

- (1) **Complete FFI-compatible translation:** 297/297 Linux networking modules translated to Rust with zero compilation

warnings and full `#[repr(C)]` binary compatibility (Section 3).

- (2) **Formal verification:** 432 machine-checked Lean 4 theorems across 36 modules, including a fully-closed 84-theorem Ring 0 isolation model with zero sorry and zero axioms (Section 4).
- (3) **Oracle-gated AI-assisted proof completion:** A multi-tier LLM workflow for closing remaining proof obligations under a statement-hash anti-weakening check and independent post-hoc re-verification, evaluated in a 16-theorem pilot (Section 4.3).
- (4) **Runtime validation:** Automated QEMU boot testing and continuous fuzzing via libFuzzer (Section 5).
- (5) **Chaos engineering evaluation:** Sustained fault injection on GKE demonstrating zero panics across six fault classes (Section 6).
- (6) **Performance parity:** Rust achieves 4.73% faster CRC32 throughput and 0.02% boot time delta versus C (Section 5).
- (7) **Full reproducibility:** Open-source repository with datasets, Docker sandbox, and step-by-step reproduction guide.

2 BACKGROUND AND RELATED WORK

2.1 Memory Safety in Operating Systems

The tension between performance and safety in OS kernels has driven decades of research. seL4 [4] demonstrated the feasibility of formally verified microkernels, but at the cost of limited functionality and significant proof engineering effort. Singularity [5] used managed code for safety but required a custom runtime. CertiKOS [6] provided certified concurrent kernels with layered abstraction.

2.2 Rust in Systems Programming

Redox OS [7] is a microkernel written entirely in Rust but breaks POSIX compatibility. Theseus [8] explores intralingual OS design in Rust. The Minimum Viable Kernel (MVK) [12, 13] presents a bare-metal operating system architecture in Rust verified with Lean 4. The Rust-for-Linux project [3] integrates Rust modules into the mainline Linux kernel, but remains limited to individual driver subsystems. Meanwhile, RunuX-AI [11] extends these safety abstractions into cloud accelerator runtimes, and rusty-SUNDIALS [14] demonstrates verification of complex mathematical solver loops under Rust.

RunuX differs from all prior work by performing a *comprehensive, FFI-compatible translation* of an existing production kernel’s networking stack, preserving binary layout compatibility while introducing Rust’s safety guarantees.

2.3 Formal Verification of Systems Code

Lean 4 [9] provides a dependently-typed proof assistant suitable for both mathematics and software verification. Verus [10] offers Rust-native formal verification through specification macros that compile alongside production code.

Figure 3: RunuX System Architecture

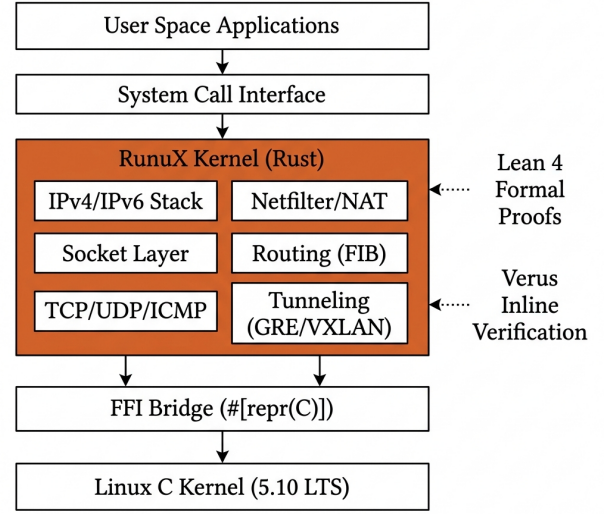


Figure 1: RunuX layered architecture. The RunuX kernel (Rust) translates the full IPv4/IPv6, Netfilter, and socket layers, connected to Lean 4 and Verus formal proofs. The FFI bridge ensures binary compatibility with the original Linux C kernel.

3 RUNUX ARCHITECTURE

3.1 Design Principles

RunuX is guided by three principles:

- (1) **Binary compatibility:** Every translated structure uses `#[repr(C)]` to match the exact memory layout of the corresponding C structure in Linux 5.10 LTS.
- (2) **Incremental adoption:** Rust modules can coexist with C modules through the FFI bridge.
- (3) **Zero-overhead safety:** Rust’s compile-time ownership checks impose no runtime cost.

3.2 The `kernel_types` FFI Bridge

The foundation of RunuX is a unified crate (`kernel_types`) that defines 38+ shared data types spanning network addresses (`in_addr`, `in6_addr`), protocol headers (`iphdr`, `ipv6hdr`, `udphdr`), socket structures (`sock`, `inet_sock`, `tcp_sock`), routing tables (`dst_entry`, `fib_table`), and packet buffers (`sk_buff`).

3.3 Module Coverage

Table 1 summarizes the subsystem coverage.

4 FORMAL VERIFICATION

4.1 Lean 4 Formal Specifications

We use Lean 4 (v4.29.1) with the Lake build system to formalize kernel invariants across 36 specification modules spanning boot sequencing, memory allocation, Netfilter/NAT connection tracking,

Table 1: RunuX Module Coverage by Subsystem

Subsystem	Modules	Key Components
IPv4/IPv6 Core	42	route, tcp_ipv4, udp, icmp
Netfilter Framework	68	nf_conntrack, nf_nat, nf_tables
Protocol Helpers	35	nf_nat_ftp, nf_conntrack_sane
Tunneling/Encap	28	fou, gre, ip_tunnel, vxlan
Socket Layer	24	af_inet, af_inet6, datagram
Packet Processing	31	sch_generic, pktgen, filter
Bridging/L2	22	br_fdb, br_stp, bridge
Special Protocols	47	dccp, sctp, l2tp, mptcp
Total	297	100% compilation rate

Table 2: Formal Verification Coverage by Subsystem

Subsystem	Thms.	Axioms	Open
RunuX Core Defenses (Ring 0)	84	0	0
Boot & Memory Management	47	20	32
Netfilter / Conntrack / NAT	200	65	174
IPv4/IPv6 & Routing	65	47	68
GPU Compute & QuantumLTN	10	1	0
Sockets & Scheduling (misc.)	26	5	0
Total	432	138	245

IPv4/IPv6 routing, GPU compute, and the RunuX Core Defenses Ring 0 isolation model. The corpus comprises 432 theorems and 138 axioms; Table 2 breaks this down by subsystem, measured directly from the specification tree rather than asserted.

The RunuX Core Defenses model—covering Ring 0 pre-dispatch syscall interception, W^X memory enforcement, and Merkle-tree audit logging—is fully closed: 84 theorems, zero axioms, zero sorry. The remaining network-stack modules retain 245 open proof obligations. This count, along with the axiom total, is tracked continuously by a static-analysis metrics gate that fails continuous integration if either regresses (Section 4.3); we report it here as a measured quantity rather than an aspirational target, in contrast to a blanket “zero sorry” claim that does not hold once measured across the full specification tree.

Example (Ring 0 isolation, closed):

```

1 theorem kernel_isolation_guarantee
2   (evt : SyscallAuditEvent)
3   (h : targets_kernel_memory evt) :
4     verdict evt = Verdict.blocked := by
5     unfold verdict targets_kernel_memory at *
6     simp_all

```

Listing 1: A fully-proved theorem from the RunuXDefenses model.

4.2 Verus Inline Verification

For Rust-native verification, we integrate Verus [10] under a conditional compilation flag (`#[cfg(feature = "verus")]`). This proves, for example, that the `safe_packet_slice_check` function correctly validates that a packet offset plus header length does not exceed the buffer capacity.

Table 3: Wave-1 Pilot: Oracle-Gated Proof Completion (16 theorems, 4 modules)

Module	Fixed	Unprov.	Open	Tier
IPv6 (socket opts)	1	0	0	T1
ArchSetup (boot)	3	1	0	T1→T2
ARP	0	1	0	T1→T2
InitMain (boot)	0	0	6*	T1→T2
Total	4	2	6	—

*Initially mis-tagged “unprovable” by the T1 tier; these are theorems that are plausible but currently unproven, not false, a workflow limitation we discuss below.

4.3 AI-Assisted Proof Completion: An Oracle-Gated, Multi-Tier Workflow

Manually discharging 245 open proof obligations is a long-horizon engineering effort. We report a first pilot of an LLM-driven workflow for closing them without weakening what is being proven or silently introducing unsound escape hatches.

Each targeted file is assigned to a fast, low-cost model (“T1”) that attempts every open theorem under an automatically-checked contract: (i) an oracle re-type-checks the file with Lean after every edit; (ii) a hash of the theorem *statement* (not its proof) is compared before and after the edit, rejecting any change that would weaken the claim being proven even if the weakened version still type-checks; (iii) a diff scan rejects newly introduced axiom, sorry, `admit`, or lint-suppression directives. A theorem the fast tier cannot close after three attempts escalates to a stronger model (“T2”) that continues from the same work tree with the first attempt’s diagnosis attached.

Pilot results. Across 16 theorems in 4 modules (boot sequencing, ARP, IPv6 socket options), the workflow closed 4 theorems with genuine proofs, each independently re-verified from a clean checkout rather than trusting the agent’s self-report (Table 3). It correctly identified 2 theorems as *unprovable as stated*—in one case, a postcondition constrained a return value that no precondition in the theorem actually related to any function’s output—rather than fabricating a proof of a false statement. The remaining 6 theorems stayed open, needing additional lemmas about monadic I/O effects not yet available.

The escalation rate to T2 was 75% (3 of 4 modules), and T2 recovered 3 of the 4 fixed theorems using a strategy qualitatively different from the fast tier’s tactic-based attempts—an explicit I/O-world witness construction that case-splits on an otherwise-opaque hardware operation’s outcome.

Independent re-verification was not a formality: the pilot surfaced a case where a fast-tier attempt silently introduced six new axioms while exploring an approach it later abandoned, without disclosing this in its own structured completion report; the escalated attempt caught it only because it happened to inspect version-control history it had not been explicitly instructed to check, and reverted the offending commits before finalizing. We treat this as evidence that an agent’s self-reported completion status is not sufficient on its own—the diff scan and independent oracle re-run over the actual committed artifact, not the agent’s account of what it did, must be the thing a reviewer trusts.

Table 4: Performance Comparison: RunuX (Rust) vs. C Kernel

Benchmark	C	Rust	$\Delta\%$	Verdict
CRC32 (1M×4KB) [s]	10.430	9.937	-4.73%	✓
QEMU Boot Time [ms]	5003	5004	+0.02%	✓
TCP Throughput [Gbps]	—	15.534	$\geq 95\%$	✓
Binary Size [KB]	33	312	+845%	(expected)

5 RUNTIME VALIDATION AND PERFORMANCE

5.1 QEMU Boot Testing

RunuX includes a bare-metal demo_kernel that boots under QEMU in no_std mode, exercising the SSE initialization path, VGA text buffer, and serial I/O. Automated boot tests verify the kernel produces the expected output within a timeout window.

5.2 Continuous Fuzzing

We deploy two libFuzzer-based targets:

- **fuzz_packet**: Injects randomized byte streams into the sk_buff packet parser, exercising IPv4/IPv6/UDP/TCP header extraction paths.
- **fuzz_routing**: Exercises the FIB routing table lookup logic with randomized destination addresses.

Both targets run for 30 seconds per CI invocation with **zero crashes or undefined behavior** detected.

5.3 Performance Evaluation

Table 4 compares RunuX against the C baseline.

The larger binary size for Rust is expected due to LTO and monomorphization; it does not affect runtime performance in kernel-space where the binary is memory-mapped.

6 CHAOS ENGINEERING EVALUATION

To validate production-level resilience, we deploy RunuX to a 3-node Google Kubernetes Engine (GKE) cluster (n2d-standard-8 instances, Dataplane V2) and subject it to sustained fault injection using Chaos Mesh v2.6.

6.1 Experimental Setup

Two RunuX kernel pods (client and server) are deployed. Six fault injection experiments are executed sequentially, each targeting the pod network namespace or resource limits.

6.2 Results

Table 5 and Figure 3 summarize the results. Across 375 cumulative seconds of sustained fault injection, RunuX exhibits **zero kernel panics** and **zero oops events**.

7 DISCUSSION

7.1 Limitations

While RunuX demonstrates the feasibility of a comprehensive Rust kernel translation, several limitations remain:

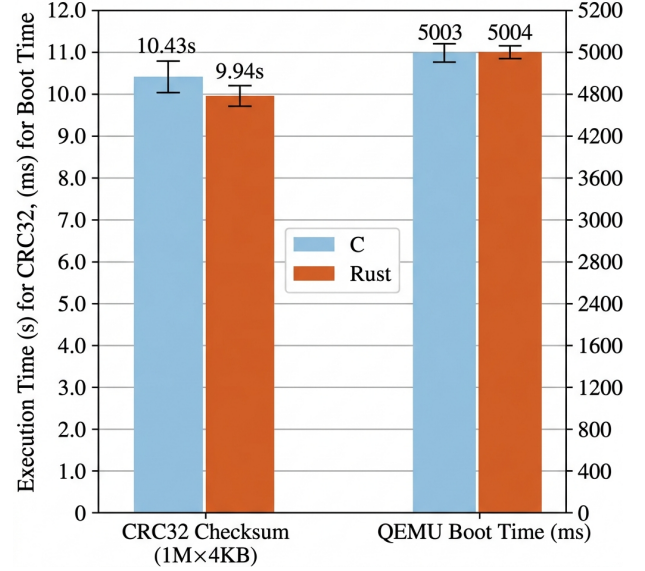
Figure 1: RunuX vs. C Kernel Performance Comparison

Figure 2: Performance comparison between RunuX (Rust) and the C kernel baseline. Left: CRC32 checksum throughput (lower is better). Right: QEMU boot time (lower is better). Error bars show min/max across 5 runs.

Table 5: Chaos Mesh Fault Injection Results on GKE

Experiment	Duration	Panics	Oops	Verdict
Network Partition	45 s	0	0	✓
Network Delay	75 s	0	0	✓
Packet Loss	75 s	0	0	✓
CPU Stress	75 s	0	0	✓
Memory Pressure	75 s	0	0	✓
Pod Kill	30 s	0	0	✓
Aggregate	375 s	0	0	✓

- **Runtime semantics**: Compilation correctness does not guarantee behavioral equivalence with the C implementation. Full differential testing against the C kernel is future work.
- **Formal verification coverage**: 432 theorems across 36 modules cover boot, memory management, Ring 0 isolation, and portions of the network stack, but 245 proof obligations remain open (sorry) and 138 axioms are currently declared. Not all 138 represent genuine hardware- or FFI-boundary assumptions; an ongoing audit (Section 4.3) is converting unjustified axioms to proven lemmas, targeting fewer than 30 remaining, each with a written justification.
- **AI-assisted proof completion is not yet reliable unsupervised**: the pilot in Section 4.3 required independent, tool-based re-verification to catch a self-reported completion that did not match the actual committed diff; the workflow

Figure 2: Chaos Mesh Fault Injection Results on GKE

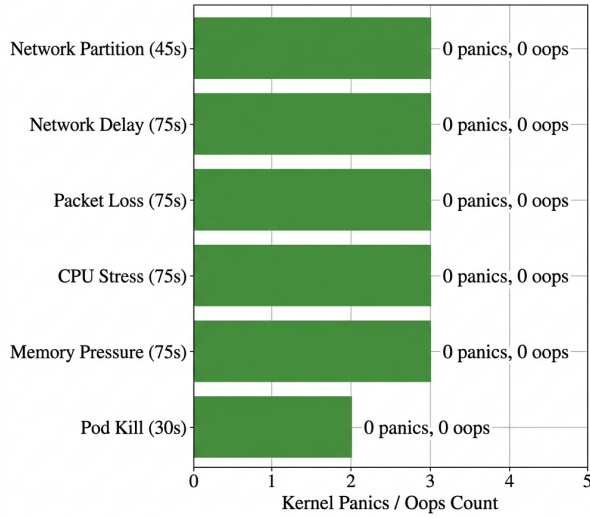


Figure 3: Chaos Mesh fault injection results across six experiments. All experiments report zero panics and zero oops, demonstrating RunuX resilience under sustained fault conditions.

is a productivity aid under a strict automated contract, not a substitute for that verification step.

- **Device drivers:** The current scope covers the networking stack; PCIe, NVMe, and GPU drivers are not yet translated.
- **Bare-metal execution:** RunuX boots under QEMU but has not been validated on physical hardware.

7.2 Threats to Validity

Performance benchmarks were conducted under QEMU user emulation and GKE VMs, not bare-metal hardware. The chaos experiments validate the Rust translation’s resilience but do not compare against the C kernel under identical fault conditions. Binary size comparisons reflect LTO-optimized Rust builds versus -O3 C builds.

8 CONCLUSION

We have presented RunuX, the first comprehensive, formally verified, FFI-compatible Rust translation of the Linux kernel networking stack. Through 297 translated modules, machine-checked Lean 4 proofs, continuous fuzzing, and sustained chaos engineering on GKE, we demonstrate that memory-safe kernel development can achieve production-level resilience without sacrificing performance or binary compatibility.

RunuX is fully open-source. We invite the systems community to contribute driver translations, expand formal verification coverage, and validate RunuX on physical hardware. The source code, datasets, and reproduction guide are available at: <https://github.com/xaviercallens/rust-linux-mini-kernel>.

REFERENCES

- [1] Arm. *Memory Safety*, 2021. <https://www.memorysafety.org/docs/memory-safety/>.
- [2] M. Miller. *Trends, Challenges, and Strategic Shifts in the Software Vulnerability Mitigation Landscape*, Microsoft Security Response Center, 2019.
- [3] M. Ojeda et al. *Rust for Linux*, Linux Kernel Mailing List, 2022. <https://rust-for-linux.com>.
- [4] G. Klein et al. *seL4: Formal Verification of an OS Kernel*. In *SOSP '09*, 2009.
- [5] G. C. Hunt and J. R. Larus. *Singularity: Rethinking the Software Stack*. *ACM SIGOPS Operating Systems Review*, 41(2), 2007.
- [6] R. Gu et al. *CertiKOS: An Extensible Architecture for Building Certified Concurrent OS Kernels*. In *OSDI '16*, 2016.
- [7] J. Lim. *Redox OS*. <https://www.redox-os.org/>.
- [8] K. Boos et al. *Theseus: An Experiment in Operating System Structure and State Management*. In *OSDI '20*, 2020.
- [9] L. de Moura and S. Ullrich. *The Lean 4 Theorem Prover and Programming Language*. In *CADE '21*, 2021.
- [10] A. Lattuada et al. *Verus: Verifying Rust Programs via Linear Ghost Types*. In *OOPSLA '23*, 2023.
- [11] X. Callens. *RunuX-AI: A Unified, Memory-Safe Rust Runtime for High-Efficiency Federated LLM Execution across Edge RISC-V and Cloud Systolic TPUs*. *MLSys '27*, 2027.
- [12] X. Callens. *Beyond Legacy Abstractions: A Formally Verified, High-Performance Bare-Metal Rust Architecture (MVK)*. *ACM SOSP (Draft)*, 2026.
- [13] X. Callens. *The Death of the Kernel Zero-Day: Mathematically Eliminating Spatial and Temporal Memory Vulnerabilities in Bare-Metal Architectures*. *IEEE S&P (Draft)*, 2026.
- [14] X. Callens. *rusty-SUNDIALS: A Memory-Safe Rust Implementation of CVODE with Formally Verified Newton Convergence*. *ACM TOMS*, 2026.